TRƯỜNG ĐẠI HỌC BÁCH KHOA – ĐẠI HỌC QUỐC GIA
THÀNH PHỐ HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN VIỄN THÔNG

**BÁO CÁO BÀI TẬP LỚN**
**XỬ LÝ SỐ TÍN HIỆU – EE2015**

# XỬ LÝ TÍN HIỆU ẢNH VÀ KỸ THUẬT SỐ (DSP)

**Lớp L01 - HK 251**
**Giảng viên hướng dẫn: THS. ĐẶNG NGỌC HẠNH**



| STT | SINH VIÊN THỰC HIỆN | MSSV |
|---|---|---|
| 1 | Phạm Thảo Nguyên | 2312380 |



*Thành phố Hồ Chí Minh – 2025*

# MỤC LỤC


CHƯƠNG 1. GIỚI THIỆU/ LÝ THUYẾT ..... 1
1.1. Sơ lược về Biến đổi fourier rời rạc DFT ..... 1
1.1.1. Phân tích miền tần số ..... 1
1.1.2. DFT (Biến đổi Fourier rời rạc) ..... 2
1.2. Biến đổi Fourier nhanh (FFT) ..... 3
1.2.1. Thuật toán FFT ..... 3
1.2.2. Phương pháp "chia để trị" trong tính toán biến đổi Fourier rời rạc (DFT) ..... 4
1.2.3. Thuật toán FFT Radix-2 ..... 6
1.2.4. Thuật toán FFT radix-4 ..... 7
1.3. Bộ lọc thông thấp và bộ lọc thông cao ..... 7
1.3.1. Bộ lọc thông thấp ..... 7
1.3.2. Bộ lọc thông cao ..... 8
CHƯƠNG 2. KIẾN TRÚC VÀ KỸ THUẬT NỀN TẢNG ..... 9
2.1. Sơ đồ hệ thống xử lý tín hiệu ..... 9
2.2. Giải thích chức năng từng khối ..... 9
2.3. So sánh phương pháp xử lý ..... 10
CHƯƠNG 3. PHƯƠNG PHÁP/ MÔ HÌNH MÔ PHỎNG ..... 11
3.1. Mục tiêu mô phỏng ..... 11
3.2. Mô hình mô phỏng ..... 11
3.3. Chỉ số đánh giá chất lượng ..... 11
3.3.1. PSNR (Peak Signal-to-Noise Ratio) ..... 11
3.3.2. SSIM (Structural Similarity Index Measure) ..... 12
3.4. Kết quả mong đợi ..... 13
CHƯƠNG 4. GIỚI THIỆU VỀ OPENCV VÀ NUMPY ..... 14
4.1. OpenCV là gì ..... 14
4.2. Các khả năng chính của OpenCV ..... 14
4.3. Tại sao OpenCV (Python) lại thống trị ..... 15
4.4. NumPy là gì ..... 15
4.5. Mối quan hệ "Bất khả phân ly" ..... 15
4.6. Tại sao mối quan hệ này lại mạnh mẽ ..... 16
4.7. Kết luận ..... 16
CHƯƠNG 5. MÔ PHỎNG VÀ ĐÁNH GIÁ KẾT QUẢ ..... 17
5.1. Ảnh 1 ..... 17
5.2. Ảnh 2 ..... 19
5.3 Ảnh 3 – Ảnh Gumball ..... 20
5.5. Ảnh 4 ..... 21
5.5. So sánh tổng hợp và nhận xét ..... 22

**5.6. Thảo luận ảnh hưởng tham số** ..... 22
**5.6.1. Tần số lấy mẫu / mật độ điểm ảnh (Fs / sampling density)** ..... 22
**5.6.2. Số bit lượng tử (quantization bits)** ..... 22
**5.6.3. Độ dài và loại cửa sổ (window) khi dùng FFT** ..... 23
**5.6.5. Hệ số tốc độ học μ trong bộ lọc thích nghi (LMS)** ..... 23
**5.6.5. Thử nghiệm quét tham số (Parameter sweep) và báo cáo** ..... 24
**5.6.6. Các lưu ý thực nghiệm khác** ..... 24
**5.7. Bài Code và Kết quả** ..... 24
**CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN** ..... 26
**TÀI LIỆU THAM KHẢO** ..... 27

# CHƯƠNG 1. GIỚI THIỆU/ LÝ THUYẾT

Biến đổi Fourier nhanh (FFT) là một thuật toán cực kì hiệu quả để chuyển đổi một tín hiệu rời rạc miền thời gian sang miền tần số dựa trên biến đổi Fourier rời rạc (DFT). Phép biến đổi DFT phân tích một dãy các số thành các thành phần ở các tần số khác nhau. Nó được ứng dụng trong nhiều lĩnh vực khác nhau nhưng tính toán trực tiếp từ định nghĩa thường quá chậm trong thực tế. FFT là một cách để đạt được cùng kết quả như DFT nhưng nhanh hơn nhiều: tính DFT của N điểm trực tiếp theo định nghĩa đòi hỏi O(N 2 ) phép tính, trong khi FFT tính ra cùng kết quả đó trong O(N log N) phép tính.

Thuật toán FFT phổ biến nhất là thuật toán FFT Cooley-Tukey. Đây là một thuật toán chia để trị dùng đệ quy để chia bài toán tính DFT có kích thước N=N1N2, thành nhiều bài toán tính DFT nhỏ hơn có kích thước N1 và N2. Dạng phổ biến nhất của thuật toán Cooley-Tukey là chia biến đổi thành hai nửa kích thước N/2 ở mỗi bước (vì vậy chỉ dùng được cho kích thước là lũy thừa của 2, còn gọi là thuật toán cơ số 2), nhưng bất kì cách phân tích ra thừa số nào cũng đều có thể dùng được. Mặc dù ý tưởng cơ bản là đệ quy, khi lập trình, người ta thường sắp xếp lại thuật toán để tránh đệ quy.

FFT có thể thực hiện theo hai hướng: Giải thuật phân chia miền thời gian và giải thuật phân chia miền tần số.

Ở đây chúng ta đi sâu vào giải thuật phân chia miền tần số.

## 1.1. Sơ lược về Biến đổi fourier rời rạc DFT:

### 1.1.1. Phân tích miền tần số:

Phân tích tần số liên quan đến việc phân giải tín hiệu thành các thành phần tần số (hình sin) của nó. Quá trình này có ba ứng dụng chính trong DSP:

Tính toán phổ tần số của tín hiệu.

Triển khai phép chập một cách hiệu quả bằng FFT.

Mã hóa dạng sóng (như tiếng nói, hình ảnh) để truyền tải và lưu trữ hiệu quả.

Biến đổi Fourier Thời gian Rời rạc (DTFT): Là biến đổi Fourier của tín hiệu thời gian rời rạc năng lượng hữu hạn x(n):

$$X(\omega)=\sum_{n=-\infty}^{\infty} x(n)e^{-j\omega n}$$

### 1.1.2. DFT (Biến đổi Fourier rời rạc):

#### *1.1.2.1. DFT là gì?*

Biến đổi Fourier Rời rạc (DFT) được sử dụng để phân tích tần số của tín hiệu rời rạc x[n]. Về mặt toán học, DFT của một dãy x[n] có độ dài L (với L ≤N).

$$\boxed{X(k) \equiv X(\frac{2\pi k}{N}) = \sum_{n=0}^{L-1} x(n) e^{-j2\pi kn/N}} \qquad \text{(N-point DFT)}$$

DFT thường được máy tính hoặc bộ xử lý DSP tính toán bằng công thức (N điểm):

Trong đó:

X(k): giá trị phức của thành phần tần số k.

x(n): tín hiệu đầu vào trong miền thời gian

N: là số mẫu tín hiệu

Các giá trị X[k] này là các mẫu tần số đều đặn của phổ DTF liên tục X(ω) của tín hiệu tại các điểm tần số $\omega_k = \frac{2\pi k}{N}$

$$x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) e^{j2\pi kn/N}, \qquad n = 0,1,\ldots,N-1 \qquad \text{(IDFT)}$$

IDFT: Biến đổi Fourier rời rạc ngược: Cho phép khôi phục lại tín hiệu:

#### *1.1.2.2. Tính tuần hoàn:*

Tuy nhiên, trong trường hợp tổng quát, các mẫu DFT này đại diện cho phổ của một dãy tín hiệu tuần hoàn (xp[n]) với chu kỳ N, chứ không phải duy nhất là dãy ban đầu x[n].

Khi tín hiệu ban đầu x[n] có độ dài vô hạn, dãy tuần hoàn xp[n] được tạo ra bởi DFT là một phiên bản bị răng cưa (aliasing) của x[n] trong miền thời gian.

$$x_p[n] = \sum_{l=-\infty}^{\infty} x[n-lN]$$

DFT có tính tuần hoàn: X[k+N]=X[k]

x[n+N]=x[n]

Điều này thể hiện việc DFT xem tín hiệu như chu kỳ lặp lại với chu kỳ N.

Nhờ vậy, DFT là nền tảng cho các kỹ thuật như phân tích phổ (spectral analysis), lọc số (digital filtering) và FFT (Fast Fourier Transform).

#### *1.1.2.3. DFT cho tín hiệu hữu hạn:*

Trong trường hợp phổ biến khi tín hiệu đầu vào x[n] là hữu hạn độ dài L (L≤N), quá trình tuần

hoàn hóa (hoặc lấy mẫu phổ DTF) đơn giản chỉ là việc lặp lại tuần hoàn của dãy x[n] sau mỗi N mẫu, tức là xp[n] là phiên bản tuần hoàn của x[n]:

$$x_p[n]=x[n], \text{ với } 0 \le n \le N-1$$

***1.1.2.4. 2D DFT:***

Trong xử lý ảnh, tín hiệu x là một ma trận 2D x(m, n) (ảnh). 2D DFT được sử dụng để phân tích các biến thể không gian (spatial frequency) theo cả hai chiều ngang và dọc.

Công thức 2D TFT là một quá trình phân tách (separable), cho phép tính toán dễ dàng bằng cách áp dụng 1D DFT lần lượt trên các chiều của ảnh (N1 là số hàng, N2 là số cột):

$$X(k_1,k_2)=\sum_{m=0}^{N_1-1}\left[\sum_{n=0}^{N_2} x(m,n)W_{N_2}^{nk_2}\right]W_{N1}^{mk_1}$$

Công thức này thể hiện rõ sự tương tự 1D ⟷ 2D: áp dụng biến đổi Fourier 1D dọc theo một chiều (các cột) và sau đó áp dụng biến đổi Fourier 1D dọc theo chiều còn lại (các hàng) của kết quả.

Ứng dụng DFT: DFT là cốt lõi của các ứng dụng DSP trong xử lý ảnh thông qua định lý Convolution. DFT cho phép chuyển các phép toán tính toán phức tạp (chập) sang các phép toán đơn giản (nhân) trong miền tần số, là nền tảng cho việc sử dụng FFT 2D để tăng tốc độ xử lý ảnh.

## 1.2. Biến đổi Fourier nhanh (FFT) :

### 1.2.1. Thuật toán FFT:

Biến đổi Fourier Nhanh FFT là một thuật toán hiệu quả để tính toán Biến đổi Fourier rời rạc (DFT).

DFT là công cụ cơ bản để chuyển đổi một tín hiệu từ miền thời gian sang miền tần số, cho phép chúng ta phân tích các thành phần sóng sin cấu thành nên tín hiệu đó. Tuy nhiên, việc tính toán DFT trực tiếp có độ phức tạp là O(N2) (với N là số mẫu của tín hiệu), khiến nó trở nên kém hiệu quả và tốn thời gian tính toán khi N lớn.

Để khắc phục hạn chế này, thuật toán FFT đã được phát triển. Nó khai thác các tính chất đối xứng và tuần hoàn của nhân biến đổi (twiddle factor) để giảm đáng kể độ phức tạp tính toán xuống chỉ còn O(N log2N).

Mục đích chính của FFT là giảm đáng kể độ phức tạp tính toán so với việc tính DFT trực tiếp: Để tính tất cả N giá trị của DFT, công thức chuẩn yêu cầu N2 phép nhân phức và N(N-1) phép cộng phức.

Số lượng phép toán với N=2r.

Nếu độ dài dữ liệu N là lũy thừa của 2 (N=2r), thuật toán FFT (radix-2) sẽ có r = log2(N) giai đoạn (stages).

Phép nhân phức (complex 'x'): $\frac{N}{2}\log_2 N$

Phép cộng phức (complex '+'): $N\log_2 N$

Việc giảm từ N2 xuống N log2 N là rất lớn, đặc biệt khi N lớn.

FFT khai thác tính chất của hệ số phức $W_N=e^{-j2\pi/N}$, còn được gọi là "Twiddle factor". Các tính chất được khai thác bao gồm:

Tính đối xứng (Symmetry): $W_N^{k+\frac{N}{2}}=-W_N^k$

Tính tuần hoàn (periodiciti): $W_N^{k+N}=W_N^k$

Thuật toán Biến đổi Fourier Nhanh (FFT) đóng vai trò cốt lõi trong chủ đề Image DSP Sampler (1D ⟷ 2D analogy) vì nó cho phép tính toán 2D DFT của ảnh một cách hiệu quả, là nền tảng cho lọc ảnh nhanh (Fast Image Filtering). FFT là phương pháp tối ưu hóa để tính DFT, và nó được ứng dụng trong 2D thông qua tính chất tách biệt (separability) của 2D DFT.

Ví dụ như cho bài toán: Cho hai dãy a = (a0, a1, a2,..., an-1) và b = (b0, b1, b2,...., bn-1).

Tính dãy c = (c0, c1, c2, ..., c2n-2) được cho bởi công thức $c_k=\sum a_j b_{k-j}$ với k từ 0 đến 2n-2. Dãy c được định nghĩa như trên được gọi là tích chập của hai dãy a và b.

Ý tưởng: Biểu thức của dãy  phía trên gợi lại cho chúng ta về phép nhân đa thức. Cụ thể, định nghĩa:

$A(z) = a_0z^0+a_1z^1+...+a_{n-1}z^{n-1}$

$B(z) = b_0z^0+b_1z^1+...+b_{n-1}z^{n-1}$

thì c chính là dãy hệ số của đa thức C(z)= A(z).B(z)

Có thể thấy rằng việc tính đa thức C theo định nghĩa sẽ có độ phức tạp thời gian O(n2), không đủ nhanh khi độ dài n khá lớn, ta cần một hướng tiếp cận khác. Giả sử ta tính được giá trị của A và B tại m điểm khác nhau:

A(z0), A(z1), ... , A(zm-1)

B(z0), B(z1), ... , B(zm-1)

thì giá trị của C tại các điểm tương ứng là: C(zj) = A(zj). B(zj).

Ý tưởng của thuật toán FFT là chọn ra một tập điểm z0, z1, z2, ..., zm-1 sao cho ta có thể tính nhanh giá trị của đa thức A và B trên đó, đồng thời có thể khôi phục được đa thức C dựa trên C(z0), C(z1), ..., C(zm-1) .

### 1.2.2. Phương pháp "chia để trị" trong tính toán biến đổi Fourier rời rạc (DFT):

Việc xây dựng các thuật toán hiệu quả cho Biến đổi Fourier Rời rạc (DFT) là một yêu cầu cấp thiết, và nền tảng cho sự phát triển này chính là kỹ thuật "Chia để trị" (Divide-and-Conquer). Phương pháp này hoạt động bằng cách phân tách một DFT có kích thước lớn N điểm thành nhiều DFT có kích thước nhỏ hơn. Nhóm các thuật toán hiệu quả dựa trên nguyên tắc này được gọi chung là thuật toán FFT.

Phương pháp "chia để trị" (Divide and Conquer) là nguyên tắc toán học được áp dụng để tối ưu hóa việc tính toán Biến đổi Fourier Rời rạc (DFT), tạo nên thuật toán Biến đổi Fourier Nhanh (FFT).

Nguyên tắc"chia để trị" được áp dụng trong thuật toán Decimation In Time (DIT) FFT để giảm đáng kể số lượng phép toán khi tính toán DFT.

Chia: Tín hiệu đầu vào x[n] được chia thành 2 chuỗi con có độ dài N/2: chuỗi các mẫu chẵn ($x_e[n]=x[2n]$) và chuỗi các mẫu lẻ: ($x_0[n]=x[2n+1]$).

Trị: Tính DFT cho từng chuỗi con $X_{chẵn}[k]$ và Xlẻ[k].

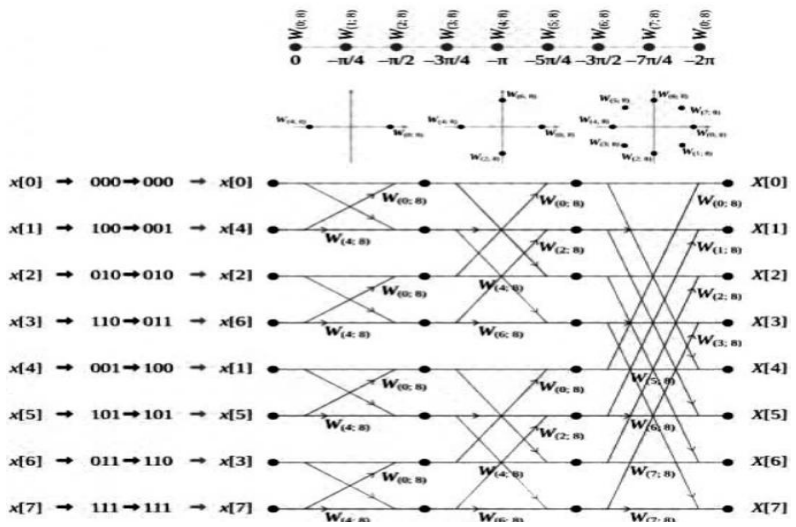
Kết hợp: Kết quả X[k] cuối cùng được tính bằng cách kết hợp 2 DFT N/2 - điểm này với hệ số xoay ($W_N^k$):

$$X[k]=X_{chẵn}[k]+W_N^k X_{lẻ}[k]$$

$$X[k+N/2]=X_{chẵn}[k]-W_N^k X_{lẻ}[k]$$

Lặp lại: Quá trình chia này lặp lại đệ quy cho đến khi chỉ còn DFT 2-điểm đơn giản (khối butterfly)

Phương pháp “chia để trị” cho phép chuyển giao hiệu quả 1D ⟷ 2D. Phương pháp này là lý do giúp việc phân tích phổ không gian của ảnh (vốn là ma trận lớn) trở nên khả thi và nhanh chóng



trên các bộ xử lý tín hiệu số.

### 1.2.3. Thuật toán FFT Radix-2:

Thuật toán FFT Cơ số 2 là một ứng dụng kinh điển của phương pháp "Chia để trị" khi kích thước DFT N là lũy thừa của 2 (tức là N=2r).

Cơ chế hoạt động của Radix-2 là phân chia dãy tín hiệu đầu vào theo thời gian (Decimation-in-Time - DIT). Ta chia DFT N điểm ban đầu thành hai DFT có kích thước nhỏ hơn là N/2 điểm. Việc lựa chọn tham số phân tách là L=2 và M=N/2.

Dãy dữ liệu gốc x(n) được phân tách thành hai dãy con, tương ứng với các mẫu có chỉ số chẵn và chỉ số lẻ:

Dãy chẵn: $f_1(n)=x(2n)$

Dãy lẽ: $f_2(n)=x(2n+1)$

Công thức DFT N điểm có thể được phân rã thành tổng của hai DFT N/2 điểm của f1(n) và $f_2(n)$ như sau:

$$X(k)=\sum_{m=0}^{N/2-1} x(2m)W_N^{2mk}+\sum_{m=0}^{N/2-1} x(2m+1)W_N^{k(2m+1)}$$

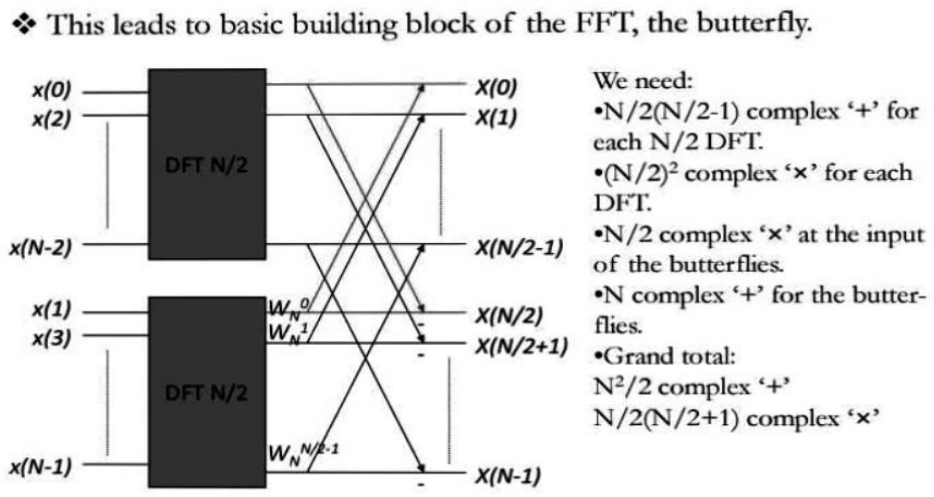
Sử dụng tính chất $W_N^2=W_{N/2}$ và nhân tử $W_N^k$ cho dãy lẻ, ta có thể biểu diễn X(k) qua hai DFT. $N/2$ điểm của F1(n) và F2(n): $X(k)=F_1(k)+W_N^kF_2(k)$

Thuật toán FFT Radix-2 là thuật toán "chia để trị" cụ thể được sử dụng để tính DFT 2D trong xử lý ảnh một cách hiệu quả nhất khi kích thước ảnh là lũy thừa của 2 (N=2r).Nó là khối xây dựng cơ bản cho việc thực hiện Lọc Ảnh Nhanh (Fast Image Filtering) trong miền tần số.

Do 2D DFT có tính chất tách biệt, thuật toán Radix-2 1D được áp dụng lặp lại theo hai bước: Một là hàng: Áp dụng thuật toán Radix-2 FFT cho từng hàng của ma trận ảnh. Hai là cột: Áp dụng lại thuật toán Radix-2 FFT cho từng cột của ma trận kết quả sau bước 1.

Các thành phần của thuật toán Radix-2 DIT:

Giảm độ phức tạp: Thuật toán Radix-2 chuyển đổi độ phức tạp tính toán 1D từ O(N2) (DFT trực tiếp) xuống O(N log2 N). Cấu trúc Butterfly: Là khối tính toán cơ bản (DFT 2-điểm)



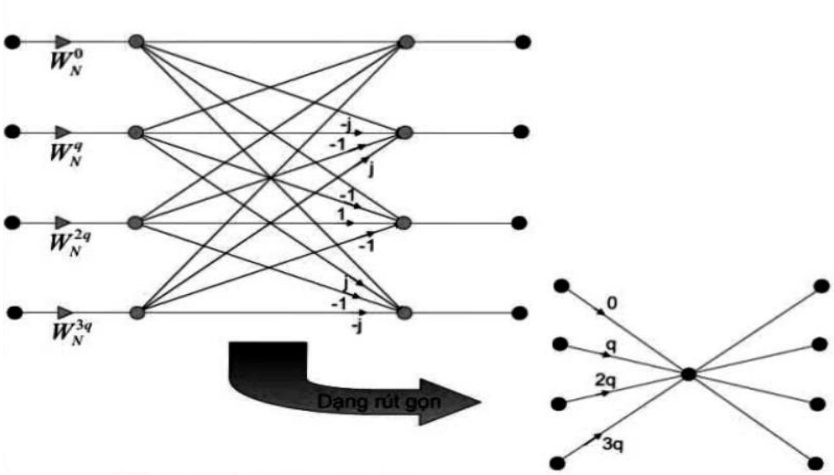
của thuật toán radix-2. Nó thực hiện phép tính X[k] và X[k+N/2] chỉ với 1 phép nhân phức và hai phép cộng/trừ phức. Cấu trúc butterfly cho phép thuật toán khai thác tính chất đối xứng và tuần hoàn của hệ số xoay $W_N^k$.

Giai đoạn: Đối với $N=2^r$ điểm, thuật toán Radix 2-điểm có tổng cộng r=log2(N) giai đoạn tính toán.

Sắp xếp dữ liệu: Thuật toán DIT Radix-2 thường yêu cầu dữ liệu đầu vào (các điểm ảnh trên một hàng hoặc cột) phải được sắp xếp lại theo thứ tự đảo bit (bit reverse ordering) trước khi bắt đầu tính toán. Điều này cho phép thực hiện tính toán tại chỗ (in-place) để tiết kiệm bộ nhớ.

### 1.2.4. Thuật toán FFT radix-4:

Thay vì chia DFT N-điểm thành hai DFT N/2-điểm (như Radix-2), thuật toán Radix-4 chia DFT N-điểm thành bốn DFT N/4-điểm nhỏ hơn. Thuật toán này chỉ áp dụng khi số điểm N là lũy thừa của 4 (ví dụ: N=64, 256, 1024,...). Khối cơ bản (Butterfly): Khối butterfly của Radix-4 có bốn đầu vào và bốn đầu ra.




Thuật toán Radix-4 DIF phân tách phương trình DFT thành bốn tổng:

$$X(k)=\sum_{n=0}^{\frac{N}{4}-1} x(n)+x(n+\frac{N}{4})(-j)^k+x(n+\frac{N}{2})(-1)^k+x(n+\frac{3N}{4})(j)^k W_N^{kn}$$

Trong đó, các phép nhân với (-j)k, (-1)k, (j)k là các hệ số xoay đặc biệt.

Ưu điểm so với radix-2: Thực hiện ít phép nhân hơn và giảm số tầng→giúp giải quyết vấn đề tăng băng thông hoạt động của chip khi bậc N tăng lên, cho phép xử lý nhanh hơn đồng thời thiết kế Radix-4 thường dẫn đến độ trễ (latency) thấp hơn, làm cho nó phù hợp hơn cho các ứng dụng DSP thời gian thực.

## 1.3. Bộ lọc thông thấp và bộ lọc thông cao:

### 1.3.1. Bộ lọc thông thấp:

Là loại bộ lọc cho phép các thành phần tần số thấp đi qua trong khi làm suy giảm các thành phần tần số cao.

Chức năng và ứng dụng:

Bộ Tiền lọc Chống chồng phổ (Antialiasing Prefilter): Trong quá trình chuyển đổi A/D, các tín hiệu trong thực tế có thể không bị giới hạn băng tần. Chúng phải được lọc bằng một bộ lọc thông thấp (LPF) trước khi lấy mẫu. Bộ tiền lọc lý tưởng có tần số cắt Fmax = Fs/2.

Bộ Tái tạo Lý tưởng (Ideal Reconstructor): Bộ tái tạo lý tưởng hoạt động như một LPF với tần số cắt bằng Tần số Nyquist Fs/2.

Đáp ứng tần số: Một bộ tái tạo lý tưởng (LPF) H(F) có đáp ứng tần số.

$$H(F)=\begin{cases} T\ , F\epsilon \left[\frac{-F_s}{2};\frac{F_s}{2}\right] \\ 0\ , \text{khác} \end{cases}$$

### 1.3.2. Bộ lọc thông cao:

HPF cho phép các thành phần tần số cao đi qua và làm suy giảm các thành phần tần số thấp.

Ứng dụng thực tế:

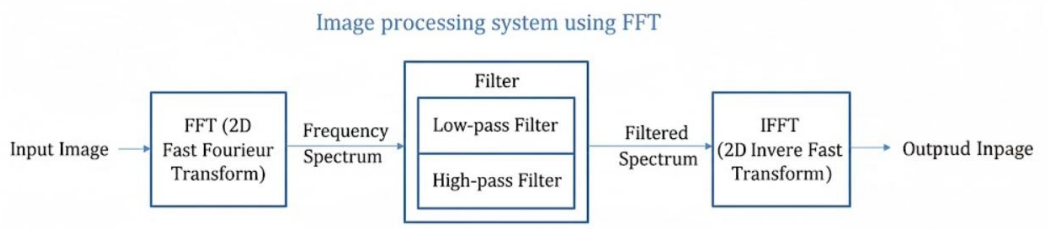
Xử lý âm thanh (lọc nhiễu, nén tín hiệu, khôi phục giọng nói).

Xử lý ảnh (lọc 2D, khử nhiễu, phát hiệu biên, PSNR/SSIM).

Tín hiệu y sinh (EEG, ECG) và tín hiệu thời gian khác.

# CHƯƠNG 2. KIẾN TRÚC VÀ KỸ THUẬT NỀN TẢNG

## 2.1. Sơ đồ hệ thống xử lý tín hiệu:




## 2.2. Giải thích chức năng từng khối:

**Chuyển đổi Miền Thời gian ↔ Miền Tần số:**

Việc chuyển đổi giữa miền Thời gian và miền Tần số là nền tảng của Xử lý Tín hiệu Số (DSP), cho phép phân tích và thao tác tín hiệu/hệ thống một cách hiệu quả.
Miền Thời gian → Miền Tần số:

Chức năng của khối này là phân tích một tín hiệu rời rạc x(n) hoặc đáp ứng xung h(n) từ dãy các mẫu thời gian sang phổ tần số tương ứng X(k) hoặc H(k):

Phân giải Tín hiệu: Dùng để phân giải tín hiệu thành các thành phần tần số cơ bản (các sóng sin/cos), giúp xác định các tần số chủ đạo.

Phân tích Hệ thống: Cho phép phân tích cấu trúc của hệ thống tuyến tính bất biến theo thời gian (LTI) thông qua việc khảo sát các đặc điểm phổ của nó, như vị trí cực (poles) và zero, từ đó đánh giá tính ổn định và đáp ứng tần số của bộ lọc.

Miền Tần số → Miền Thời gian: Khối này thực hiện thao tác ngược lại, tổng hợp lại tín hiệu/đáp ứng xung trong miền thời gian từ phổ tần số đã được biến đổi hoặc xử lý.

Xác định Đáp ứng Xung (h(n)): Dùng để xác định đáp ứng xung h(n) của bộ lọc hoặc hệ thống bằng cách áp dụng IDFT cho hàm truyền đạt tần số H(k) (hoặc H(z)).
Tính Đầu ra Tín hiệu (y(n)): Dùng để tính toán tín hiệu đầu ra y(n) của hệ thống bằng cách thực hiện IDFT/IFFT trên phổ đầu ra Y(k).

Khối Liên kết: Phép chập trong Miền Tần số.

Mối liên kết này khai thác tính chất cơ bản của Biến đổi Fourier: Phép chập tuyến tính trong miền Thời gian tương đương với phép nhân trong miền Tần số.

Mục tiêu: Cho phép thực hiện thao tác lọc tuyến tính (chập y(n) = x(n) * h(n)) một cách hiệu quả về mặt tính toán.

Cơ chế: Bằng cách sử dụng DFT và đặc biệt là thuật toán FFT, phép chập $O(N^2)$ được thay thế bằng quy trình ba bước: FFT, Nhân, và IFFT, giảm độ phức tạp xuống còn $O(N \log_2 N)$.

**2.3. So sánh phương pháp xử lý:**

So sánh phương pháp xử lý.

Bộ lọc trong Miền Thời gian (Chập Trực tiếp) với Bộ lọc trong Miền Tần số (sử dụng FFT)

Trong DSP, việc áp dụng một bộ lọc FIR (Finite Impulse Response) cho một tín hiệu đầu vào x(n) được thực hiện thông qua phép chập tuyến tính (linear convolution). Phép chập y(n) = x(n) * h(n) có thể được tính toán bằng hai phương pháp chính:

| Tiêu chí | Bộ lọc trong Miền Thời gian (Chập Trực tiếp) | Bộ lọc trong Miền Tần số (Sử dụng FFT) |
|---|---|---|
| Phép toán cơ bản | Chập tuyến tính trực tiếp: $y(n)=\sum_{k=0}^{M} x(k)h(n-k)$ | Nhân phức trong miền tần số: $Y(k)=X(k)H(k)$ |
| Quy trình tính toán | Thực hiện phép chập trực tiếp. | 3 bước:<br>1. DFT/FFT của x(n) và h(n).<br>2. Nhân kết quả: $Y(k)=X(k)H(k)$.<br>3. IDFT/IFFT của Y(k). |
| Độ phức tạp | $O(L.M)$ hoặc $O(N^2)$ (nếu $L \approx M \approx N$). | $O(N \log_2 N)$ (được chi phối bởi FFT/IFFT). |
| Yêu cầu Zero Padding | Không cần (đã tính chập trực tiếp). | Bắt buộc (đối với chập tuyến tính). Kích thước DFT/FFT tối thiểu phải là $N \geq L_x+L_h-1$, trong đó $L_x$ và $L_h$ là độ dài của tín hiệu và đáp ứng xung. |

# CHƯƠNG 3. PHƯƠNG PHÁP/ MÔ HÌNH MÔ PHỎNG

## 3.1. Mục tiêu mô phỏng:

Mục tiêu cốt lõi của mô hình mô phỏng là chứng minh tính phổ quát của các nguyên lý Xử lý Tín hiệu Số khi mở rộng từ miền 1D sang 2D.

Thể hiện rõ sự tương đồng giữa xử lý tín hiệu số 1 chiều (DSP 1D) và xử lý ảnh số 2 chiều (Image DSP).

Nguyên lí: Mỗi mô phỏng được thiết kế để minh họa nguyên lý DSP 1D (lấy mẫu, lượng tử hóa, lọc số, biến đổi Fourier) và phiên bản mở rộng 2D trong miền không gian ảnh.

Nền tảng: Mô phỏng các phép toán DSP bằng Python.

## 3.2. Mô hình mô phỏng:

Mô hình mô phỏng được xây dựng dựa trên quy trình xử lý tín hiệu số chuẩn trong DSP, gồm các khối: lấy mẫu (sampling), lượng tử hóa (quantization), xử lý số (filtering / FFT) và tái tạo (reconstruction).

Khối chức năng:

Lấy mẫu (sampling): Lấy mẫu tín hiệu 1D (thời gian) và Lấy mẫu ảnh 2D (không gian). (Áp dụng định lý Nyquist)

Lượng tử hóa: Giảm số bit mã hóa (độ sâu bit) cho tín hiệu 1D và ảnh 2D. Xác định bước lượng tử $Q=\frac{R}{2^B}$ và gây ra sai số lượng tử.

Xử lý số: Lọc số (FIR/IIR) hoặc Phân tích/Biến đổi Fourier (FFT). Áp dụng phép chập.

Tái tạo: Chuyển đổi mẫu số trở lại thành tín hiệu/ảnh. Sử dụng bộ lọc thông thấp (Analog Lowpass Filter)

Các bước này được thực hiện song song cho cả tín hiệu 1D và ảnh 2D nhằm thể hiện sự tương đồng trong nguyên lý hoạt động.

## 3.3. Chỉ số đánh giá chất lượng:

### 3.3.1. PSNR (Peak Signal-to-Noise Ratio):

Phạm vi áp dụng: Đánh giá tín hiệu 2D (ảnh)

Mục đích: Chỉ số tiêu chuẩn cho chất lượng ảnh sau xử lý (Lấy mẫu/ Lượng tử/ Nén).

Công thức sai số bình phương trung bình (MSE):

$$MSE=\frac{1}{m.n}\sum_{i=0}^{m-1}\sum_{j=0}^{n-1}(o(i,j)-D(i,j))^2$$

trong đó O(i, j) là giá trị pixel tại vị trí (i, j) của ảnh gốc, và D(i,j) là giá trị tương ứng của ảnh đã xử lý.

Thông số MSE (Mean Squared Error) trong DSP dùng để đo lường độ chính xác của một hệ thống xử lý tín hiệu bằng cách đánh giá mức độ sai lệch giữa tín hiệu gốc và tín hiệu đã xử lý.
Công thức PSNR:

$$\text{PSNR}=10\log_{10}\left(\frac{\text{MAX}_I^2}{\text{MSE}}\right)$$

Trong đó: MAX là giá trị cực đại có thể có của pixel trong ảnh (peak signal value).
Thông số PSNR (Peak Signal-to-Noise Ratio) trong DSP là một chỉ số đánh giá chất lượng tín hiệu tái tạo hoặc nén, bằng cách so sánh mức năng lượng cực đại của tín hiệu gốc với mức nhiễu gây sai lệch. PSNR càng cao thì chất lượng tín hiệu càng tốt.

**3.3.2. SSIM (Structural Similarity Index Measure):**

Phạm vi áp dụng: Đánh giá tín hiệu 2D
Mục đích: Đánh giá độ tương đồng cấu trúc giữa ảnh gốc và ảnh đã xử lý/tái tạo (thay vì chỉ dựa trên sai số điểm ảnh đơn giản).

Công thức SSIM:

$$\text{SSIM}(x,y)=\frac{(2\mu_x\mu_y+C_1)(2\sigma_{xy}+C_2)}{(\mu_x^2+\mu_y^2+C_1)(\sigma_x^2+\sigma_y^2+C_2)}$$

Trong đó: $\mu_x$, $\mu_y$: trung bình của ảnh x và y

$\sigma_x^2$, $\sigma_y^2$: phương sai của ảnh x và y

$\sigma_{xy}$: hiệp phương sai giữa x và y

Thông số SSIM (Structural Similarity Index) trong DSP là một chỉ số đánh giá mức độ tương đồng về cấu trúc giữa hai tín hiệu, thường dùng để so sánh chất lượng tín hiệu gốc và tín hiệu đã xử lý hoặc tái tạo. SSIM càng gần 1 thì chất lượng càng tốt.

SNR/SQNR (Signal-to-Noise Ratio)
Đánh giá tín hiệu 1D: Đánh giá chất lượng của quá trình lượng tử hóa. Thể hiện tỷ lệ tín hiệu trên tạp âm do lỗi lượng tử gây ra:

$$SNR_{dB}\approx 6B\ dB$$

RMS Error (Root Mean Square Error)
Đánh giá tín hiệu 1D & 2D: Sai số trung bình gốc bình phương. Đánh giá độ lớn trung bình của sai số lượng tử ($e_{rms}=\frac{Q}{\sqrt{12}}$) hoặc sai số sau khi tái tạo/xử lý.

PSNR (Peak Signal-to-Noise Ratio:

Đánh giá tín hiệu 2D: Chỉ số tiêu chuẩn cho chất lượng ảnh sau xử lý (Lấy mẫu/ Lượng tử/ Nén).

SSIM (Structural Similarity Index Measure)

Đánh giá tín hiệu 2D: Đánh giá độ tương đồng cấu trúc giữa ảnh gốc và ảnh đã xử lý/tái tạo (thay vì chỉ dựa trên sai số điểm ảnh đơn giản).

### 3.4. Kết quả mong đợi:

Sau khi thực hiện các mô phỏng, kết quả mong đợi là minh họa được rõ ràng sự tương đồng giữa xử lý tín hiệu số 1D và xử lý ảnh số 2D thông qua các hiện tượng và hiệu ứng đặc trưng của từng khối DSP.

Các kết quả phải thể hiện được cả định tính (hình dạng tín hiệu, thay đổi trực quan của ảnh) và định lượng (qua các chỉ số PSNR, MSE, SSIM).

Chứng minh được tính phổ quát của các nguyên lý DSP (ví dụ: cách giảm B bit làm tăng $\sigma_q^2$ trong 1D, đồng thời làm giảm PSNR và chất lượng ảnh 2D).

# CHƯƠNG 4. GIỚI THIỆU VỀ OPENCV VÀ NUMPY

## 4.1. OpenCV là gì?

OpenCV (viết tắt của Open Source Computer Vision Library) là một thư viện mã nguồn mở hàng đầu thế giới, tập trung vào các ứng dụng thị giác máy tính (computer vision), xử lý ảnh (image processing) và học máy (machine learning) theo thời gian thực.

Ra đời lần đầu vào năm 1999 bởi Intel, OpenCV ban đầu được viết bằng C/C++. Ngày nay, nó hỗ trợ đa nền tảng (Windows, macOS, Linux, Android, iOS) và cung cấp giao diện cho nhiều ngôn ngữ lập trình, bao gồm C++, Java, và đặc biệt là Python, nơi nó trở nên cực kỳ phổ biến.

Mục tiêu chính của OpenCV là cung cấp một hạ tầng thị giác máy tính dễ sử dụng, hiệu suất cao để giúp các nhà phát triển xây dựng các ứng dụng phức tạp một cách nhanh chóng.

## 4.2. Các khả năng chính của OpenCV:

OpenCV là một thư viện khổng lồ với hơn 2.500 thuật toán đã được tối ưu hóa. Các chức năng của nó bao trùm gần như mọi khía cạnh của thị giác máy tính:

Đọc, ghi và hiển thị ảnh/video: Có khả năng đọc và ghi hầu hết các định dạng ảnh (JPEG, PNG,...) và video (MP4, AVI,...), cũng như thu nhận hình ảnh trực tiếp từ webcam.

Xử lý ảnh cơ bản: thay đổi không gian màu (ví dụ: từ RGB sang HSV hoặc Xám); các phép biến đổi hình học (xoay, co giãn, lật ảnh).; các bộ lọc hình ảnh (làm mờ, làm nét, lọc thông thấp, lọc thông cao); phân ngưỡng (Thresholding) và tạo ảnh nhị phân; các phép toán hình thái học (Morphological Operations)

Phân tích và Trích xuất Đặc trưng: phát hiện biên cạnh (Canny, Sobel); phát hiện đặc trưng (SIFT, SURF, ORB) để ghép ảnh hoặc theo dõi đối tượng; tìm và vẽ đường viền (Contours).

Thị giác máy tính nâng cao: Phát hiện đối tượng (Object Detection): Sử dụng các kỹ thuật cổ điển (như Haar Cascades cho nhận diện khuôn mặt) hoặc các mô hình Deep Learning hiện đại (tích hợp với các framework như TensorFlow, PyTorch).

Theo dõi đối tượng (Object Tracking): Theo dõi chuyển động của một vật thể qua các khung hình video.

Tái tạo 3D và Lập thể (Stereo Vision): Ước tính chiều sâu từ nhiều camera.

Tích hợp Học máy: Cung cấp các thuật toán ML cơ bản như K-Nearest Neighbors (KNN), Support Vector Machine (SVM) để phân loại.

### 4.3. Tại sao OpenCV (Python) lại thống trị?

Sự kết hợp giữa OpenCV và Python đã tạo nên một "cặp đôi hoàn hảo" vì:

Dễ học, dễ viết: Ngôn ngữ Python trong sáng, dễ hiểu, giúp việc viết các thuật toán xử lý ảnh phức tạp trở nên đơn giản hơn rất nhiều so với C++.

Hiệu suất cao: Mặc dù giao diện là Python, phần lõi của OpenCV vẫn được viết bằng C/C++ đã tối ưu hóa, đảm bảo tốc độ xử lý nhanh.

Cộng đồng lớn: Có một cộng đồng người dùng và nhà phát triển khổng lồ, đồng nghĩa với vô số tài liệu, hướng dẫn và sự hỗ trợ.

Tương thích: Nó hoạt động hoàn hảo với các thư viện khoa học dữ liệu khác của Python, và thư viện quan trọng nhất chính là NumPy.

### 4.4. NumPy là gì?

NumPy (viết tắt của Numerical Python) là thư viện cơ bản và cốt lõi cho tính toán khoa học trong Python. Bản thân NumPy không phải là thư viện xử lý ảnh, mà là thư viện dùng để làm việc với mảng (array) đa chiều.

Thành phần trung tâm của NumPy là đối tượng ndarray (N-dimensional array), một cấu trúc dữ liệu mạnh mẽ cho phép:

- Lưu trữ và thao tác với các khối dữ liệu lớn (như vector, ma trận) một cách hiệu quả về bộ nhớ.
- Thực hiện các phép toán toán học trên toàn bộ mảng (vector hóa) mà không cần dùng vòng lặp for, giúp tốc độ tính toán nhanh hơn hàng trăm lần so với code Python thuần.

### 4.5. Mối quan hệ "Bất khả phân ly":

Đây là khái niệm quan trọng nhất cần hiểu khi làm việc với OpenCV trong Python:

Một hình ảnh trong OpenCV (Python) CHÍNH LÀ một mảng NumPy.

Khi bạn dùng hàm cv2.imread() để đọc một bức ảnh, OpenCV không trả về một định dạng "ảnh" độc quyền nào cả. Nó trả về một đối tượng numpy.ndarray.

Mối quan hệ này được thể hiện như sau:

Ảnh Xám (Grayscale): Là một mảng NumPy 2 chiều (2D)

Kích thước: (chiều_cao, chiều_rộng)

Mỗi phần tử trong mảng là một giá trị số nguyên (thường là 8-bit, từ 0 đến 255) biểu thị cường độ sáng.

Ảnh Màu (Color): Là một mảng NumPy 3 chiều (3D).

Kích thước: (chiều_cao, chiều_rộng, 3)

Lớp thứ 3 (kênh màu) biểu thị các giá trị màu. Lưu ý quan trọng: OpenCV mặc định sử dụng thứ tự BGR (Blue, Green, Red), thay vì RGB (Red, Green, Blue) như các thư viện khác (ví dụ: Matplotlib).

**4.6. Tại sao mối quan hệ này lại mạnh mẽ?**

Việc ảnh là mảng NumPy mở ra vô số khả năng:

Truy cập và Thao tác trực tiếp: Bạn không cần các hàm getPixel() hay setPixel() chậm chạp. Bạn có thể truy cập trực tiếp vào một điểm ảnh bằng cú pháp slicing của NumPy.

Lấy pixel tại hàng 50, cột 100: pixel = image[50, 100]

Lấy kênh màu Đỏ (Red) của pixel đó: red_value = image[50, 100, 2]

Cắt một vùng ảnh (Region of Interest - ROI): roi = image[100:200, 300:400]

Sử dụng toàn bộ sức mạnh của NumPy: Bất kỳ phép toán nào bạn có thể thực hiện trên mảng NumPy, bạn đều có thể thực hiện trên ảnh.

Tăng độ sáng toàn bộ ảnh: brighter_image = np.clip(image + 50, 0, 255)

Đảo ngược ảnh (âm bản): inverted_image = 255 - image

Tạo ảnh đen từ đầu: black_image = np.zeros((480, 640, 3), dtype="uint8")

Tương thích hệ sinh thái: Dữ liệu (ảnh) của bạn có thể được đưa thẳng vào các thư viện khoa học dữ liệu khác cũng dùng NumPy, như:

Matplotlib / Seaborn: Để vẽ biểu đồ hoặc hiển thị ảnh.

SciPy: Chứa các hàm xử lý ảnh và tín hiệu nâng cao (như lọc, biến đổi Fourier).

Scikit-learn: Để đưa dữ liệu ảnh vào các mô hình học máy.

**4.7. Kết luận:**

OpenCV cung cấp các thuật toán và chức năng (cái "WHAT") để thực hiện các tác vụ thị giác máy tính. NumPy cung cấp cấu trúc dữ liệu (cái "HOW") để lưu trữ và thao tác với hình ảnh một cách hiệu quả nhất trong môi trường Python.

Khi bạn sử dụng OpenCV trong Python, bạn không chỉ đang học về xử lý ảnh, mà bạn còn đang học cách sử dụng NumPy. Nắm vững cách thao tác mảng NumPy chính là chìa khóa để khai thác toàn bộ tiềm năng của OpenCV.

# CHƯƠNG 5. MÔ PHỎNG VÀ ĐÁNH GIÁ KẾT QUẢ

Trong chương này, các kết quả mô phỏng được trình bày và phân tích chi tiết cho bốn bộ ảnh khác nhau. Các phép lọc được sử dụng gồm lọc thông thấp (Low-pass Filter), lọc thông cao (High-pass Filter), và các bộ lọc tương ứng trong miền tần số (FFT). Các chỉ số PSNR và SSIM được dùng để đánh giá định lượng chất lượng ảnh sau khi lọc, giúp kiểm chứng lý thuyết đã học về xử lý tín hiệu số trong miền không gian và tần số.

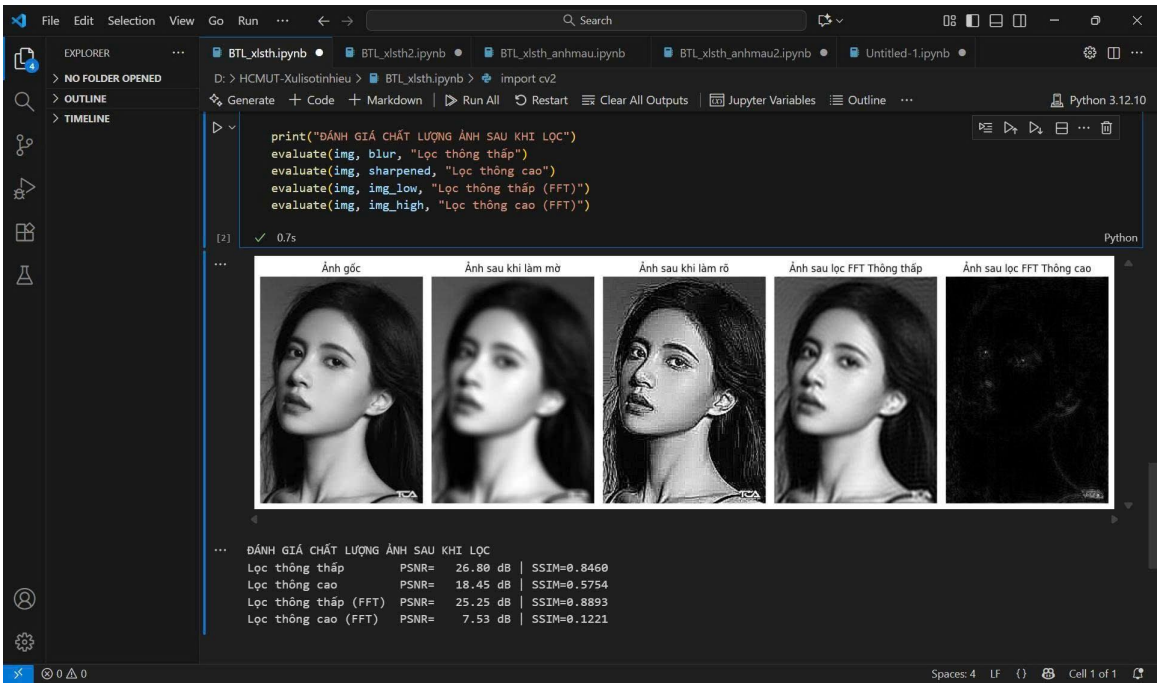
Đánh giá định lượng sử dụng các chỉ tiêu:

**PSNR (Peak Signal-to-Noise Ratio):** là đo mức độ sai khác cường độ điểm ảnh so với ảnh gốc.

**SSIM (Structural Similarity Index):** đánh giá mức độ tương đồng cấu trúc giữa ảnh gốc và ảnh xử lý.

PSNR càng cao tái tạo chi tiết càng giống ảnh gốc , ssim nằm trong khoảng [-1;1] (thường nằm trong khoảng [0;1]) , càng gần 1 tái tạo hình ảnh càng tốt và dưới 0 thì hình ảnh chỉ giữ được chi tiết nhưng bị đảo cực sáng hoặc tối , mất toàn bộ nền .

## 5.1. Ảnh 1:




***Hình 5.1***

Bảng 5.1. Các giá trị PSNR và SSIM cho ảnh 1:

Lọc thông thấp : PSNR = 26.80 dB, SSIM = 0.8460

Lọc thông cao : PSNR = 18.45 dB, SSIM = 0.5754

Lọc thông thấp (FFT) : PSNR = 25.25 dB, SSIM = 0.8893

Lọc thông cao (FFT) : PSNR = 7.53 dB, SSIM = 0.1221

*Phân tích:*

Ảnh sau khi áp dụng lọc thông thấp trở nên mờ hơn nhưng nhiễu được giảm rõ rệt. Điều này phù hợp với lý thuyết vì bộ lọc này loại bỏ các thành phần tần số cao. So với ảnh gốc, ảnh sau lọc thông thấp trông mượt hơn nhưng mất đi chi tiết nhỏ và các vùng chuyển sắc trở nên đều hơn .

Trong khi đó, lọc thông cao giúp nổi bật các biên và chi tiết nhỏ, nhưng làm giảm PSNR do nhiễu tăng. Ảnh sau lọc thông cao có độ nét rõ ràng, nhưng dễ xuất hiện nhiễu biên.

Lọc trong miền tần số (thông thấp FFT) cho kết quả SSIM cao nhất , thể hiện khả năng duy trì cấu trúc của ảnh gốc tốt nhất. Nền hay vùng đồng nhất được làm mượt, nhiễu giảm vùng da/ nền trông đồng đều hơn so với ảnh gốc . Biên được giữ khá tốt ,các đường viền vật thể, khuôn mặt, đường viền quần áo vẫn nhận dạng được . Nhưng một số chi tiết rất nhỏ (sợi tóc mảnh, texture nền) bị giảm bớt so với gốc.

Lọc thông cao FFT cho kết quả tệ nhất vì chỉ giữ được các chi tiết ở biên , làm mất chi tiết ở vùng tần số thấp là vùng trung tâm .

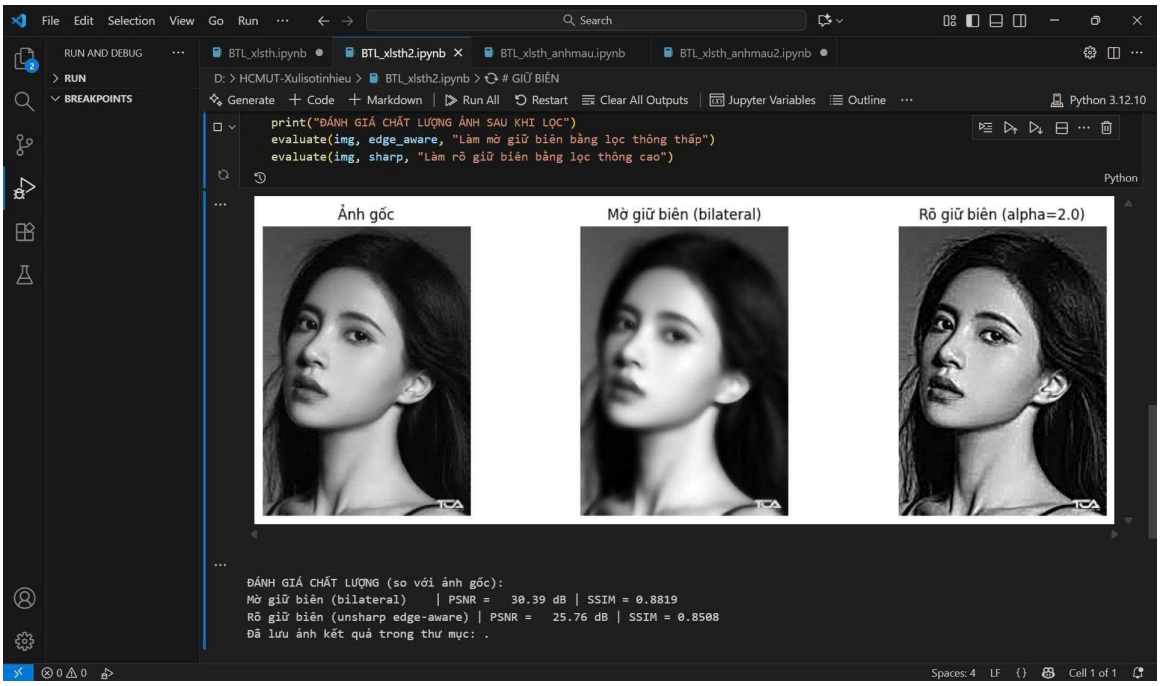
So sánh giữa lọc thông thấp và thông thấp FFT:

Lọc thông thấp FFT trực tiếp tác động lên phổ tần nên loại bỏ thành phần tần số thấp/giảm dần tần số cao theo mặt nạ lọc . Vì chứa đựng lựa chọn chính xác vùng tần số nên có thể giữ cấu trúc tổng thể tốt hơn. So với lọc thông thấp làm mượt cục bộ (kernel đơn giản) nên giảm MSE nhiều hơn ở nhiều pixel làm PSNR cao hơn nên lọc FFT có thể giữ tương quan cấu trúc vùng lớn nên SSIM cao hơn.

*Kết luận :*

Tùy theo mục đích sử dụng, lọc thông thấp phù hợp khi cần tối ưu PSNR và làm mượt nhiễu, trong khi FFT thích hợp khi yêu cầu giữ biên và đặc trưng hình học của ảnh. Việc lựa chọn phương pháp phụ thuộc vào mục tiêu của bài toán xử lý ảnh.

## 5.2. Ảnh 2:




***Hình 5.2***

Bảng 5.2. Các giá trị PSNR và SSIM cho ảnh 2:

Mờ giữ biên(bilateral) : PSNR = 30.39 dB, SSIM = 0.8819

Rõ giữ biên(unsharp edge-aware) : PSNR = 25.76 dB, SSIM = 0.8508

*Phân tích:*

Bộ lọc mờ giữ biên giúp làm mượt các vùng trong khi vẫn giữ nguyên biên, nhờ vào cơ chế trọng số phụ thuộc cả vào khoảng cách không gian và sự khác biệt cường độ điểm ảnh. PSNR và SSIM đều cao cho thấy ảnh sau lọc rất gần với ảnh gốc. So với ảnh gốc, ảnh lọc mờ giữ biên giữ nguyên hình dạng và độ mịn tự nhiên, giảm nhiễu mà không mất biên.

Bộ lọc rõ giữ biên làm tăng độ sắc nét bằng cách khuếch đại các thành phần tần số cao, dẫn đến PSNR giảm nhẹ nhưng ảnh nhìn rõ nét hơn. Kết quả này phù hợp với lý thuyết tăng cường biên trong xử lý ảnh.So với ảnh gốc , Ảnh lọc rõ giữ biên có độ sắc nét cao hơn ảnh gốc , đặc biệt ở vùng biên, song có thể làm tăng nhiễu nhẹ ở vùng đồng nhất .

## 5.3 Ảnh 3 – Ảnh Gumball:

***Hình 5.3***

Bảng 5.3. Các giá trị PSNR và SSIM cho ảnh3:

Lọc thông thấp: PSNR = 17.95 dB, SSIM = 0.7701

Lọc thông cao: PSNR = 19.83 dB, SSIM = 0.8541

Lọc thông thấp (FFT): PSNR = 11.93 dB, SSIM = 0.6333

Lọc thông cao (FFT): PSNR = 1.47 dB, SSIM = -0.0284

*Phân tích:*

Lọc thông thấp làm mất chi tiết đáng kể, khiến ảnh bị mờ và giảm độ tương phản. Ảnh gốc có độ tương phản tự nhiên và biên mềm mại, trong khi ảnh sau lọc thông thấp bị mờ khá đáng kể .

Lọc thông cao làm nổi biên tốt hơn nhưng có xu hướng khuếch đại nhiễu. Ảnh sau lọc thông cao trở nên sắc nét nhưng màu sắc bị sai lệch nhẹ .

Lọc thông thấp trong miền tần số (lọc thông thấp FFT) cho thấy méo biên và hiện tượng ringing do bộ lọc lý tưởng gây ra. Ảnh trở nên mờ trên toàn cục, độ nét giảm đáng kể. Ảnh có xu hướng bị bệt màu, các vùng màu được san phẳng. Độ tương phản giữa các biên thấp hơn đáng kể so với ảnh gốc. Những hiện tượng trên thể hiện đúng bản chất của bộ lọc thông thấp trong miền tần số là giữ lại các thành phần tần số thấp (màu, vùng phẳng) và loại bỏ hoàn toàn tần số cao (chi tiết, biên).

Lọc thông cao trong miền tần số (lọc thông cao FFT) làm hình bị méo nghiêm trọng ,chỉ giữ lại được chi tiết nhưng không giữ lại được nền , PSNR giảm rất mạnh, chứng tỏ tần số cắt không phù hợp.

So sánh lọc thông thấp và thông thấp FFT trên ảnh màu :

Lọc thông thấp là lọc trực tiếp trên ảnh bằng tích chập với kernel nên ảnh sau lọc mờ đều, chuyển sắc mượt, không gây méo hình. Còn lọc thông thấp FFT là lọc bằng cách cắt bỏ các

thành phần tần số cao trong miền tần số. Gây ra nhiễu gợn sóng, làm vùng màu loang và kém tự nhiên. Khi lọc FFT xử lý từng kênh màu RGB riêng lẽ. Khi cắt tần số cao, sự phân bố biên độ mỗi kênh thay đổi khác nhau tạo sai lệch giữa R–G–B. Mắt người rất nhạy với sai lệch này khiến màu ảnh dễ bị bệt, loang, xám lại.

*Kết luận:*

Kết luận lại , đối với ảnh màu sử dụng  lọc thông thấp sẽ tối ưu hơn do lọc thông thấp xử lý đồng đều cả 3 kênh màu (R, G, B) làm cho kết quả tự nhiên hơn với chỉ số PSNR và SSIM của lọc thông thấp  luôn cao hơn FFT cho ảnh màu.

**5.5. Ảnh 4:**

***Hình 5.4***

Bảng 5.5. Các giá trị PSNR và SSIM cho ảnh 4:

Làm mờ giữ biên :PSNR = 35.73 dB, SSIM = 0.9627
Làm rõ giữ biên: PSNR = 29.32 dB, SSIM = 0.9501

*Phân tích:*

Ảnh Minion cho kết quả rất tốt với cả hai phương pháp. Lọc thông thấp giữ biên giúp giảm nhiễu mà vẫn bảo tồn cấu trúc hình dạng, đạt PSNR và SSIM cao nhất trong toàn bộ thí nghiệm. So với ảnh gốc, ảnh Minion sau lọc thông thấp giữ biên mịn hơn nhưng vẫn rõ đường viền, không bị mờ chi tiết quan trọng.

Lọc thông cao giữ biên làm tăng độ tương phản, thể hiện rõ các chi tiết nhỏ mà không phá vỡ biên ảnh. Đây là minh chứng cho hiệu quả của các bộ lọc thích nghi (edge-preserving) trong việc duy trì chất lượng ảnh thị giác.So với ảnh gốc , ảnh sau lọc thông cao giữ biên có độ sắc nét cao, màu sắc nổi bật hơn, đặc biệt ở vùng chi tiết như mắt và quần áo.

### 5.5. So sánh tổng hợp và nhận xét:

Bảng 5.5. So sánh tổng hợp PSNR và SSIM giữa các ảnh:

Ảnh Grayscale: PSNR = 26.80–7.53, SSIM = 0.846–0.122

Ảnh Bilateral/Unsharp: PSNR = 30.39–25.76, SSIM = 0.882–0.851

Ảnh Gumball: PSNR = 19.83–1.47, SSIM = 0.854–(-0.028)

Ảnh Minion: PSNR = 35.73–29.32, SSIM = 0.963–0.950

*Nhận xét:*

Ảnh Minion có chất lượng cao nhất cả về PSNR và SSIM, trong khi ảnh Gumball cho thấy độ nhạy lớn với tần số cắt . Các kết quả đều phù hợp với lý thuyết: lọc thông thấp giảm nhiễu nhưng làm mờ ảnh, lọc thông cao làm ảnh sắc nét hơn nhưng giảm tương đồng cấu trúc, và phương pháp FFT cần được điều chỉnh mặt nạ cẩn thận.

### 5.6. Thảo luận ảnh hưởng tham số:

Trong các mô phỏng xử lý ảnh và lọc số, các tham số mô phỏng có ảnh hưởng lớn tới hiệu năng và đặc tính của phương pháp. Dưới đây là phân tích và thảo luận chi tiết về các tham số tiêu biểu: tần số lấy mẫu Fs (hoặc mật độ điểm ảnh), số bit lượng tử, kiểu và độ dài cửa sổ (window) khi làm việc với FFT, và hệ số tốc độ học μ trong các bộ lọc thích nghi (LMS).

#### 5.6.1. Tần số lấy mẫu / mật độ điểm ảnh (Fs / sampling density):

Ý nghĩa: Với ảnh, Fs tương ứng với mật độ điểm ảnh (pixels per unit) — tức là độ phân giải không gian. Tăng Fs (tức độ phân giải cao hơn) giữ lại nhiều thành phần tần số cao hơn trong ảnh, giúp các bộ lọc có cơ sở để bảo tồn chi tiết. Ngược lại, Fs thấp (ảnh có độ phân giải thấp) làm giảm dải tần có sẵn, làm tăng khả năng aliasing.

Ảnh hưởng thực nghiệm (kỳ vọng):

- Fs ↑ → chi tiết tốt hơn, PSNR thường tăng nếu thuật toán lọc giữ biên; nhưng chi phí tính toán và bộ nhớ ↑.
- Fs ↓ → nhiều chi tiết bị mất trước khi lọc, dẫn tới PSNR giảm; HPF có thể trở nên ít hiệu quả hoặc gây artefact khi cố gắng khôi phục chi tiết không tồn tại.
- Nếu dữ liệu có aliasing (do Fs thấp), cần tiền lọc chống alias (antialiasing LPF) trước khi lấy mẫu.

#### 5.6.2. Số bit lượng tử (quantization bits):

Ý nghĩa: Số bit B quyết định số mức lượng tử L = 2^B cho mỗi mẫu/giá trị pixel. Giảm B tăng sai số lượng tử (quantization noise) và làm giảm SQNR/PSNR.

Ảnh hưởng thực nghiệm (kỳ vọng):

- B ↑ → độ chính xác biên/chi tiết tăng, PSNR và SSIM cải thiện; tuy nhiên kích thước lưu trữ ↑.
- B ↓ → tăng tiếng nhiễu lượng tử, PSNR giảm, SSIM giảm; các bộ lọc HPF có thể khuếch đại nhiễu lượng tử.

Lưu ý: Khi đánh giá bằng PSNR/SSIM, cần báo rõ B dùng cho cả ảnh gốc và ảnh xử lý để so sánh công bằng.

**5.6.3. Độ dài và loại cửa sổ (window) khi dùng FFT:**

Ý nghĩa: Khi thực hiện lọc trong miền tần số bằng FFT, việc lấy mẫu khung (windowing) và kích thước N của FFT ảnh hưởng đến độ phân giải tần số và hiện tượng rò rỉ phổ (spectral leakage).

Ảnh hưởng thực nghiệm (kỳ vọng):

- N (kích thước FFT) ↑ → độ phân giải tần số ↑ (độ phân giải nhỏ nhất $\Delta f = Fs/N$ ↓), giúp thiết kế mặt nạ lọc mượt hơn; nhưng tính toán ↑.
- Cửa sổ chữ nhật (no window) → mainlobe hẹp nhưng sidelobe lớn → leakage mạnh (có thể gây ringing).
- Cửa sổ như Hamming/Hann → giảm sidelobe (giảm leakage) nhưng làm dải chính rộng hơn (giảm khả năng phân biệt hai tần số gần nhau).
- Với ảnh, lựa chọn window và zero-padding phù hợp giúp tránh artefact biên và cải thiện SSIM.

Khuyến nghị thực hành:

1) Dùng zero-padding để tăng độ phân giải tần số khi cần mặt nạ mịn.

2) Nếu gặp ringing, thử dùng cửa sổ Hamming hoặc đưa vào một filter ramp (transition band) thay vì mặt nạ lý tưởng cứng.

**5.6.5. Hệ số tốc độ học μ trong bộ lọc thích nghi (LMS):**

Ý nghĩa: Trong thuật toán LMS (Least Mean Squares), μ là bước điều chỉnh trọng số (learning rate). Nó quyết định tốc độ hội tụ và ổn định của bộ lọc thích nghi.

- Ảnh hưởng thực nghiệm (kỳ vọng):
  μ quá lớn → hội tụ nhanh nhưng có thể gây dao động/không ổn định và tăng sai số steady-state.

- μ quá nhỏ → hội tụ chậm, thời gian thích nghi kéo dài; tuy nhiên sai số cuối cùng nhỏ hơn (lower misadjustment).

Ứng dụng cho ảnh: khi dùng LMS để ước lượng hoặc tái tạo (ví dụ adaptive denoising), cần cân bằng μ để tránh khuếch đại nhiễu tạm thời.

Khuyến nghị thực hành:

- Thực hiện sweep μ trên log-scale (ví dụ μ = [1e-4, 1e-3, 1e-2, 1e-1]) để quan sát tốc độ hội tụ và PSNR/SSIM cuối cùng.
- Báo kết quả đồ thị hội tụ (PSNR theo epoch) để minh họa ảnh hưởng μ trong báo cáo.

**5.6.5. Thử nghiệm quét tham số (Parameter sweep) và báo cáo:**

Để đánh giá đầy đủ ảnh hưởng của tham số, nên thực hiện một loạt thử nghiệm quét tham số (sweep), ví dụ:

- Vary Fs / resize ảnh: thử kích thước 256×256, 512×512, 1024×1024 để thấy ảnh hưởng resolution.
- Vary quantization bits: thử B = 8, 6, 4, 2.
- Vary FFT size and window: thử N = 256, 512, 1024 với window = rectangular / hamming / hann.
- Vary LMS μ: thử log-scale μ và vẽ đồ thị hội tụ.

Kết quả cần báo cáo gồm: Bảng PSNR/SSIM cho mỗi cấu hình, đồ thị so sánh (PSNR vs parameter), và mô tả ngắn về xu hướng/hiệu ứng quan sát được. Việc này phù hợp với yêu cầu 'Sweep tham số chuẩn' trong hướng dẫn BTL.

**5.6.6. Các lưu ý thực nghiệm khác:**

- Xử lý biên (boundary handling): mirror vs zero-padding sẽ ảnh hưởng rõ tới artefact gần cạnh.
- Thang xám vs màu: đánh giá PSNR/SSIM nên thống nhất không gian màu (ví dụ so sánh trên Y channel của YCbCr khi so sánh ảnh màu).
- Lượng tử hóa trung gian: nếu lưu ảnh tạm ở dạng 8-bit giữa pipeline sẽ tạo noise không mong muốn.
- Replication reproducibility: lưu kịch bản (script) và seed ngẫu nhiên để kết quả có thể tái tạo.

**5.7. Bài Code và Kết quả:**

Link Code:

https://drive.google.com/drive/folders/1CWRp4nTx5pUfFHMnZJNVZO4ZVhiUkqGP?usp=sharing

# CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Qua các thí nghiệm, ta nhận thấy rằng các phương pháp lọc ảnh số hoạt động đúng với cơ sở lý thuyết. Lọc thông thấp giúp loại bỏ nhiễu và làm mượt ảnh nhưng gây mất chi tiết. Lọc thông cao làm tăng độ sắc nét nhưng khuếch đại nhiễu. Các phép lọc trong miền tần số (FFT) cho phép điều khiển chính xác phổ tần nhưng cần lựa chọn tham số thích hợp để tránh hiện tượng méo ảnh.

Đặc biệt, các phương pháp lọc giữ biên (Bilateral, Edge-preserving) cho kết quả tốt nhất, với PSNR và SSIM cao, đồng thời duy trì được hình dạng và biên vật thể – rất phù hợp cho các ứng dụng như ảnh y sinh, thị giác máy tính và phục hồi ảnh.

Hướng phát triển: Trong tương lai, có thể mở rộng nghiên cứu sang các phương pháp lọc thích nghi (adaptive filtering), lọc phi tuyến (nonlinear filtering), hoặc kết hợp học sâu (deep learning-based enhancement) để cải thiện hơn nữa chất lượng và tốc độ xử lý ảnh.

# TÀI LIỆU THAM KHẢO